```
/**
 * Quieto Backend
 * ----------------
 * Minimal Node.js/Express server that
powers worldwide city search.
 *
 * Data source: GeoNames
(https://www.geonames.org) — free account
required.
 * GeoNames provides real, worldwide
population figures per city.
 *
 * SETUP:
 * 1. Create a free account at
https://www.geonames.org/login
 * 2. Activate the free webservice:
https://www.geonames.org/manageaccount
 *    (after login, click "Free Web
Services" -> Enable)
 * 3. Copy your username and put it in the
.env file (see .env.example)
 * 4. npm install
 * 5. npm start
 *
 * DEPLOY (free):
 * - Push this folder to a GitHub repo
 * - Go to render.com -> New -> Web Service
-> connect your repo
 * - Set environment variable
GEONAMES_USERNAME
```

```
 * - Build command: npm install | Start
command: npm start
 */

const express = require("express");
const cors = require("cors");

const app = express();
app.use(cors());
app.use(express.json());

const PORT = process.env.PORT || 3001;
const GEONAMES_USERNAME =
process.env.GEONAMES_USERNAME || "demo";

// Country area data (km²) for a selection
of countries, used to estimate
// national average density when city-level
area isn't available.
// This is a fallback only — GeoNames
doesn't return city area directly.
const APPROX_CITY_AREA_KM2 = {
  // A small set of known city areas (km²)
for better density accuracy.
  // Extend this list over time with cities
you care about.
  "Tokyo": 627,        // Tokyo's 23 wards
  "Paris": 105,
  "New York": 783,
  "Berlin": 891,
  "Mumbai": 603,
  "Singapore": 728,
  "Vienna": 415,
  "Dubai": 4114,
```

```
  "Barcelona": 101,
  "Lisbon": 100,
  "Porto": 41,
  "Reykjavik": 274,
  "Marrakech": 230,
  "Kyoto": 828,
};

/**
 * GET /api/search?q=cityname
 * Searches GeoNames for matching
cities/places worldwide,
 * returns name, country, coordinates,
population, and an
 * estimated population density
(people/km²) where possible.
 */
app.get("/api/search", async (req, res) =>
{
  const query = req.query.q;
  if (!query) {
    return res.status(400).json({ error:
"Missing query parameter 'q'" });
  }

  try {
    const url =
`http://api.geonames.org/searchJSON?
q=${encodeURIComponent(
      query

)}&maxRows=10&featureClass=P&orderby=popula
tion&username=${GEONAMES_USERNAME}`;
```

```
    const response = await fetch(url);
    const data = await response.json();

    if (data.status) {
      // GeoNames returns an error object
in `status` when something's wrong
      return res.status(502).json({ error:
data.status.message || "GeoNames error" });
    }

    const results = (data.geonames ||
[]).map((place) => {
      const area =
APPROX_CITY_AREA_KM2[place.name];
      const density = area ?
Math.round(place.population / area) : null;

      return {
        name: place.name,
        country: place.countryName,
        countryCode: place.countryCode,
        latitude: parseFloat(place.lat),
        longitude: parseFloat(place.lng),
        population: place.population,
        areaKm2: area || null,
        popDensity: density, // null if we
don't have area data for this city
        geonameId: place.geonameId,
      };
    });

    res.json({ results });
  } catch (err) {
    console.error(err);
```

```
    res.status(500).json({ error: "Failed
to fetch city data" });
  }
});

app.get("/", (req, res) => {
  res.json({ status: "Quieto backend is
running", endpoints: ["/api/search?
q=cityname"] });
});

app.listen(PORT, () => {
  console.log(`Quieto backend listening on
port ${PORT}`);
});
```